



Estrutura de Dados

2026.2

Professor: João Nicollas

Algoritmos de Ordenação

Os algoritmos de ordenação são um conjunto de instruções que recebem um array como entrada e organizam os itens em uma ordem específica.

E por que eles são importantes ?

Os algoritmos de ordenação são importantes em Ciência da Computação porque a ordenação pode, muitas vezes, reduzir a complexidade de um problema. Esses algoritmos têm aplicações diretas em algoritmos de busca, algoritmos de banco de dados, métodos de divisão e conquista, algoritmos de estrutura de dados e muito mais.

Algoritmos de Ordenação

Ao usar algoritmos diferentes, algumas perguntas devem ser feitas. Qual o tamanho da coleção que está sendo ordenada? Quanta memória está disponível para uso? A coleção precisa crescer?

As respostas a essas perguntas podem determinar qual algoritmo funcionará melhor para a situação.

Alguns algoritmos como a ordenação por intercalação (*Merge Sort*) podem precisar de muito espaço para serem executados, enquanto a ordenação por inserção (*Insertion Sort*) nem sempre é a mais rápida, mas não requer muitos recursos para isso.

Você deve determinar quais são os requisitos do sistema e suas limitações antes de decidir qual algoritmo usar.

Algoritmos de Ordenação

Vamos ver os seguintes algoritmos:

- Bubble Sort
- Selection Sort
- Insertion Sort
- Merge Sort
- Quick Sort

Bubble Sort

Como funciona ?

Percorre todo o vetor verificando seus elementos e o elemento adjacente, se eles estiverem ordenados segue para o próximo par, se não troque-os de lugar, essa operação é feita até que nenhuma troca possa ser feita no vetor.

Bubble Sort

Exemplo: Ordene em ordem crescente



Bubble Sort

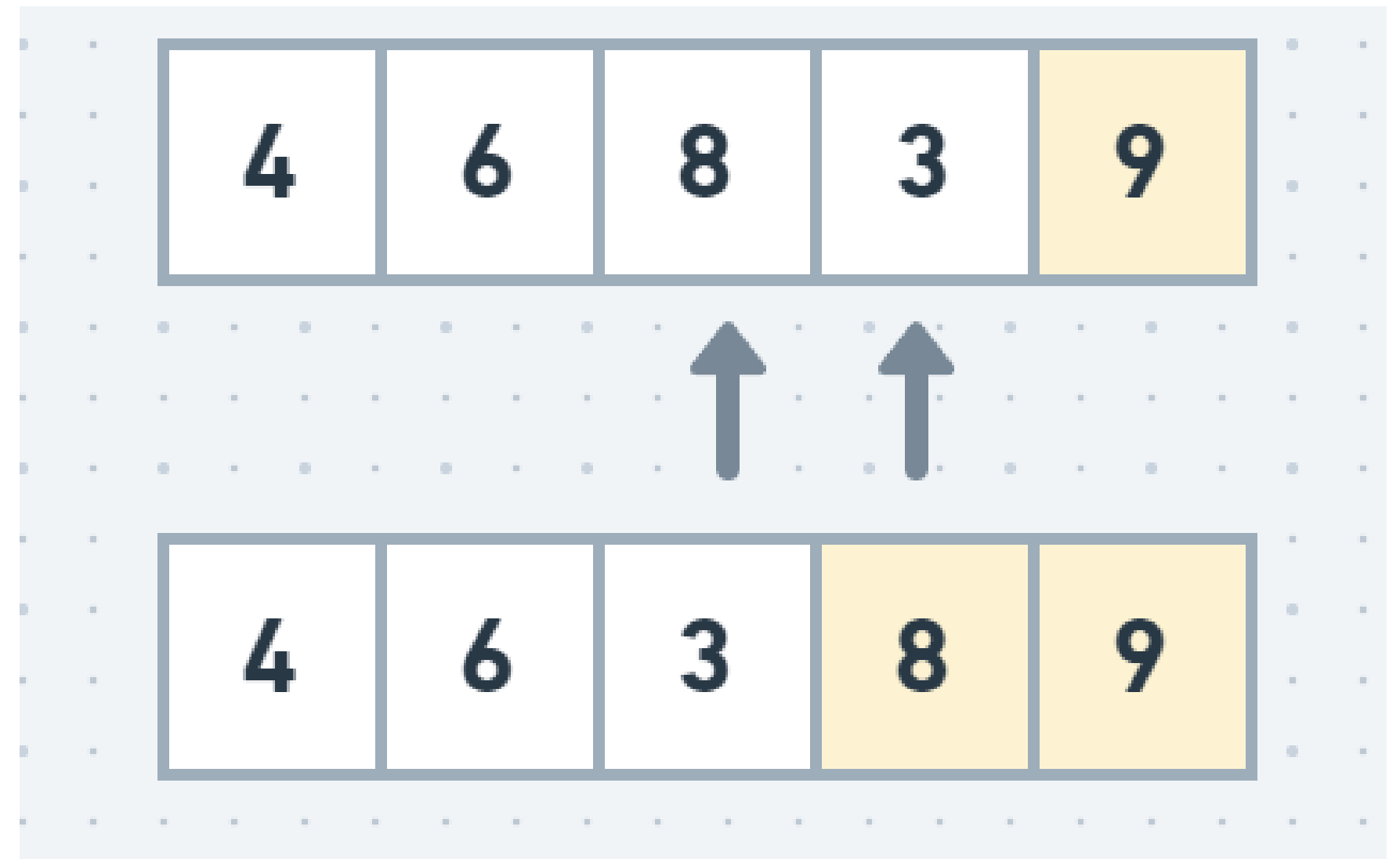
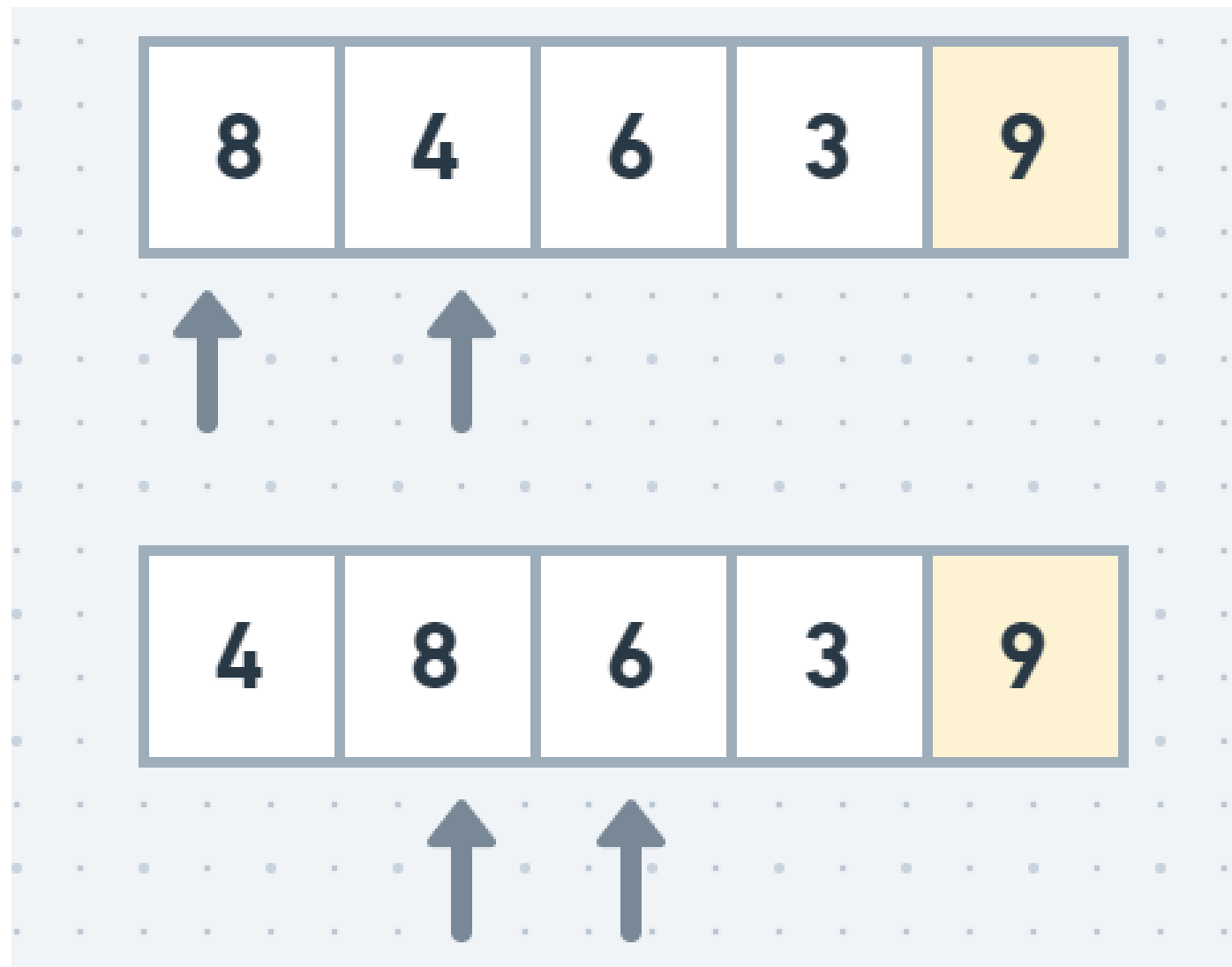
Então no final da primeira passada completa, temos o último elemento fixado sendo o maior do vetor



Logo, nas seguintes execuções, não há necessidade de verificar até o fim do vetor, apenas até o último elemento fixado.

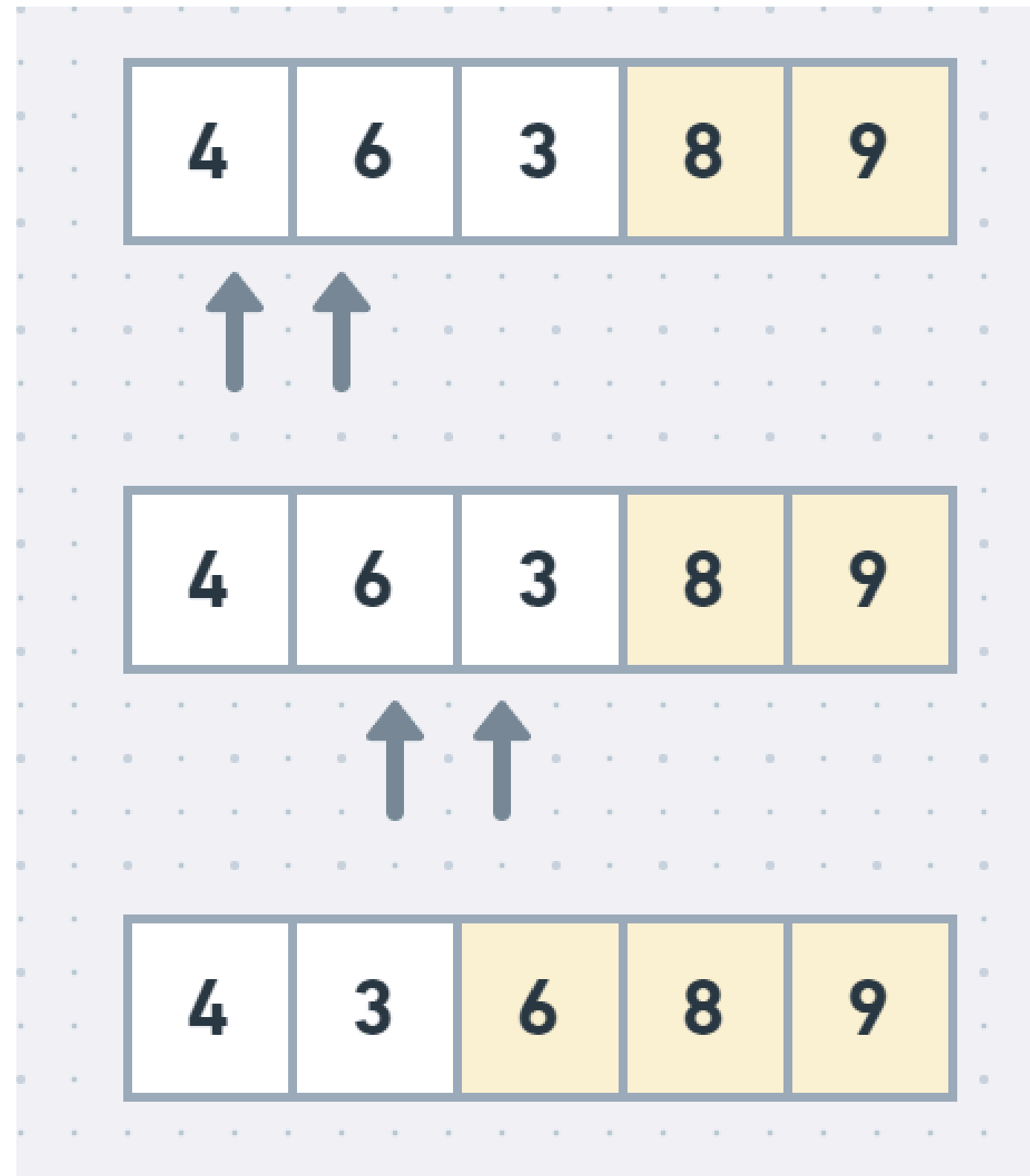
Bubble Sort

Segunda passada



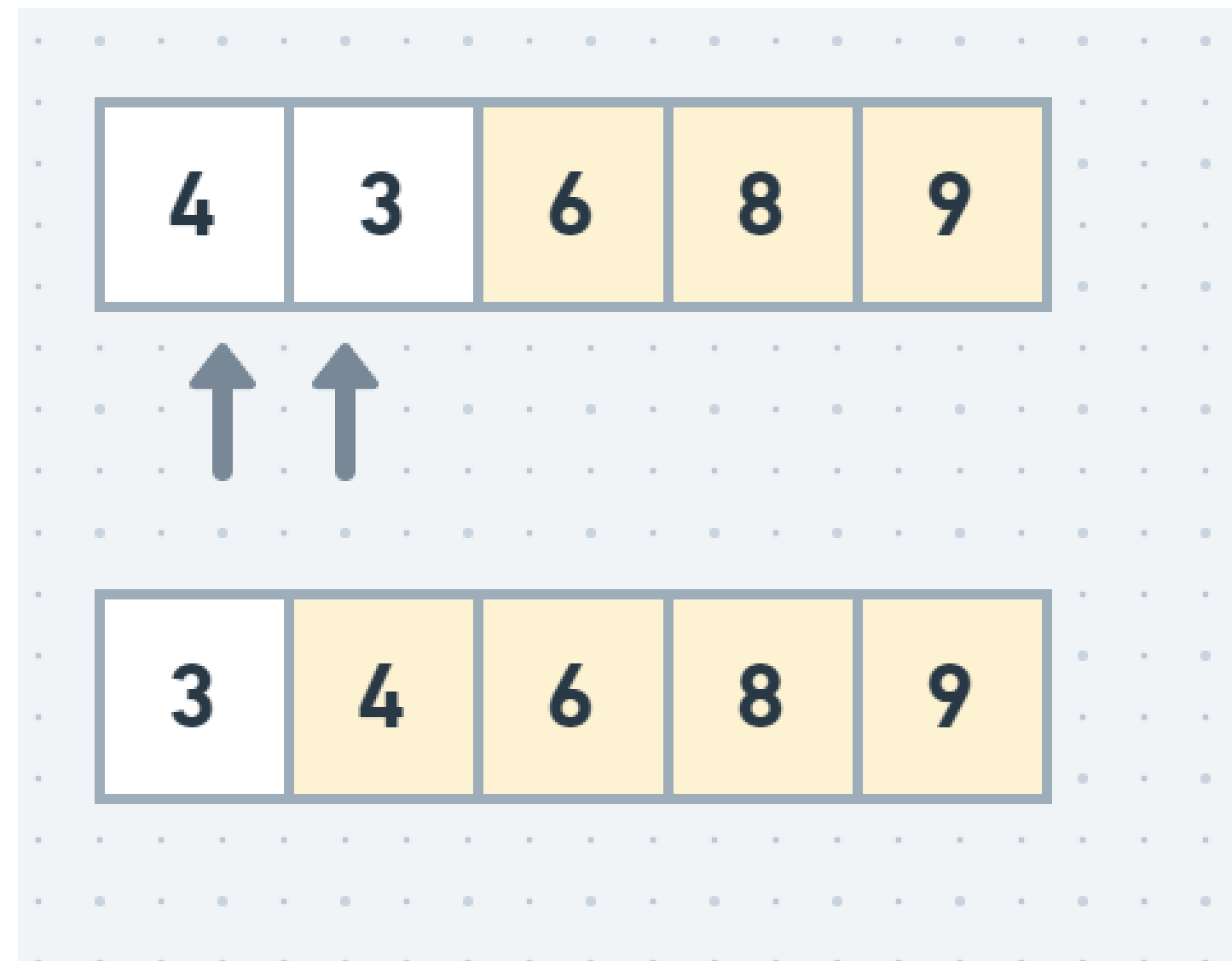
Bubble Sort

Terceira passada



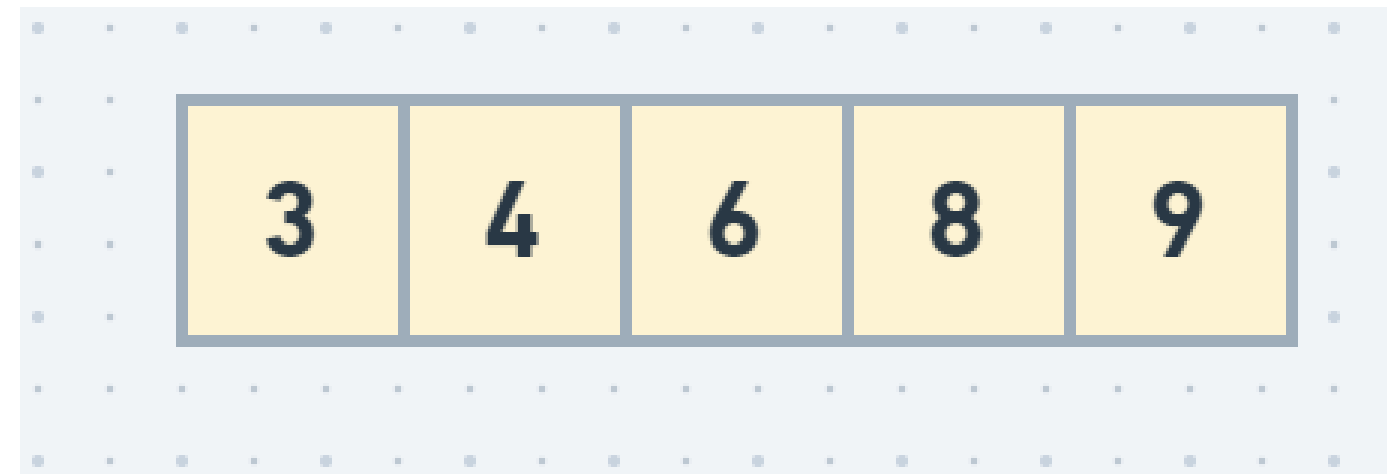
Bubble Sort

Quarta passada



Bubble Sort

Quinta passada



Bubble Sort

Observe que não precisamos ir até o final do vetor, basta irmos até onde garantimos estar ordenado.

Bubble Sort

Implementem!

Bubble Sort

Implementação

```
void bubbleSort(int vetor[], int n) {  
    for(int ult = n - 1; ult > 0; ult--) {  
        for(int i = 0; i < ult; i++) {  
            if(vetor[i] > vetor[i + 1]) {  
                int aux = vetor[i];  
                vetor[i] = vetor[i + 1];  
                vetor[i + 1] = aux;  
            }  
        }  
    }  
}
```

A variável “ult” corresponde a última posição ordenada. Logo a cada execução diminuimos o valor de ult.

Insertion Sort

Como funciona ?

Percorre todo o vetor a cada novo elemento, procurando onde ele irá se encaixar a esquerda. Então o novo elemento é adicionado e todos os elementos entre a nova posição e a original, são deslocados para a direita.

Semelhante ao modo como ordenamos cartas de um baralho.

Percorremos o vetor da esquerda para a direita e, à medida que avançamos vamos deixando as cartas (elementos) a esquerda ordenados

Insertion Sort

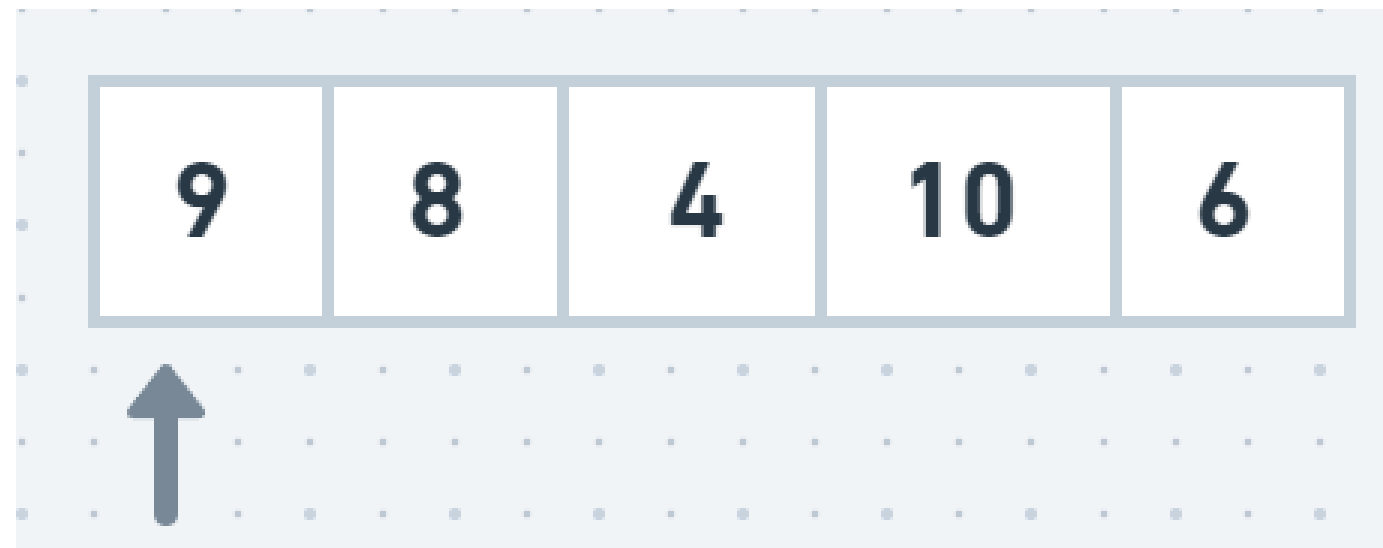
Exemplo: Ordene em ordem crescente

9 8 4 10 6

Insertion Sort

Exemplo: Ordene em ordem crescente

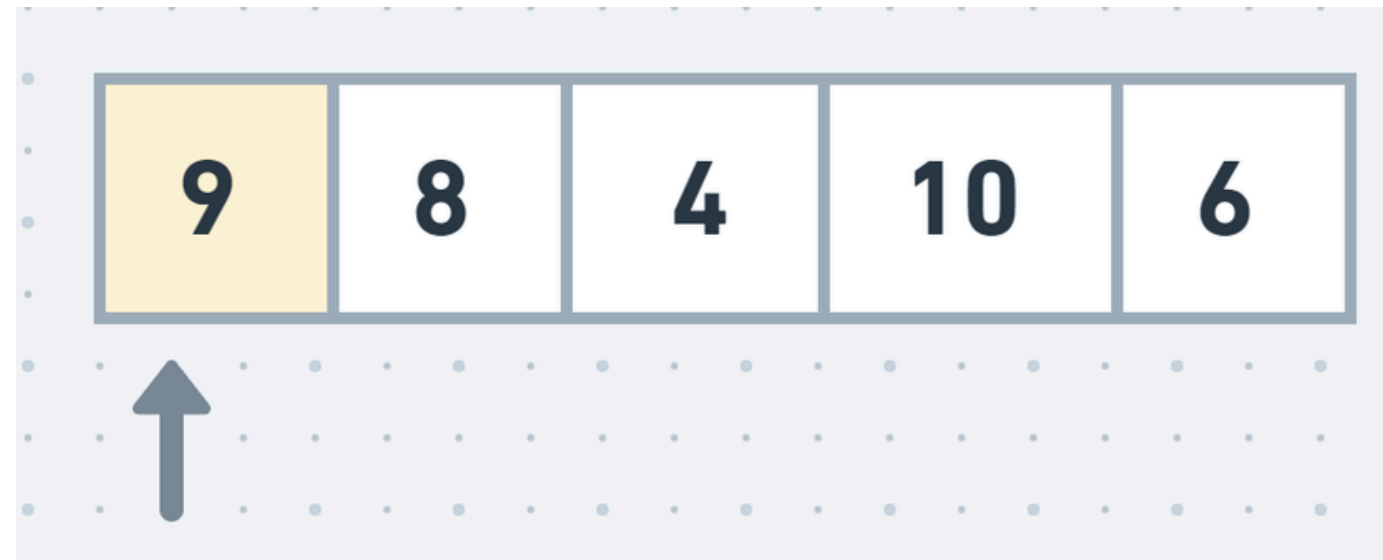
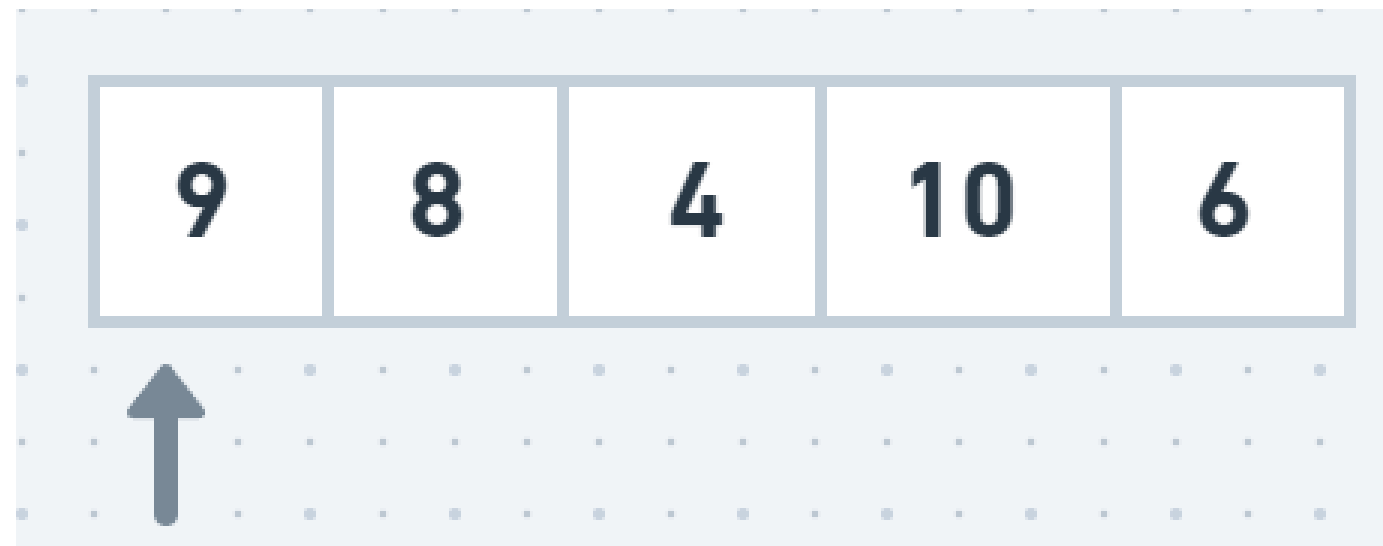
9 8 4 10 6



Insertion Sort

Exemplo: Ordene em ordem crescente

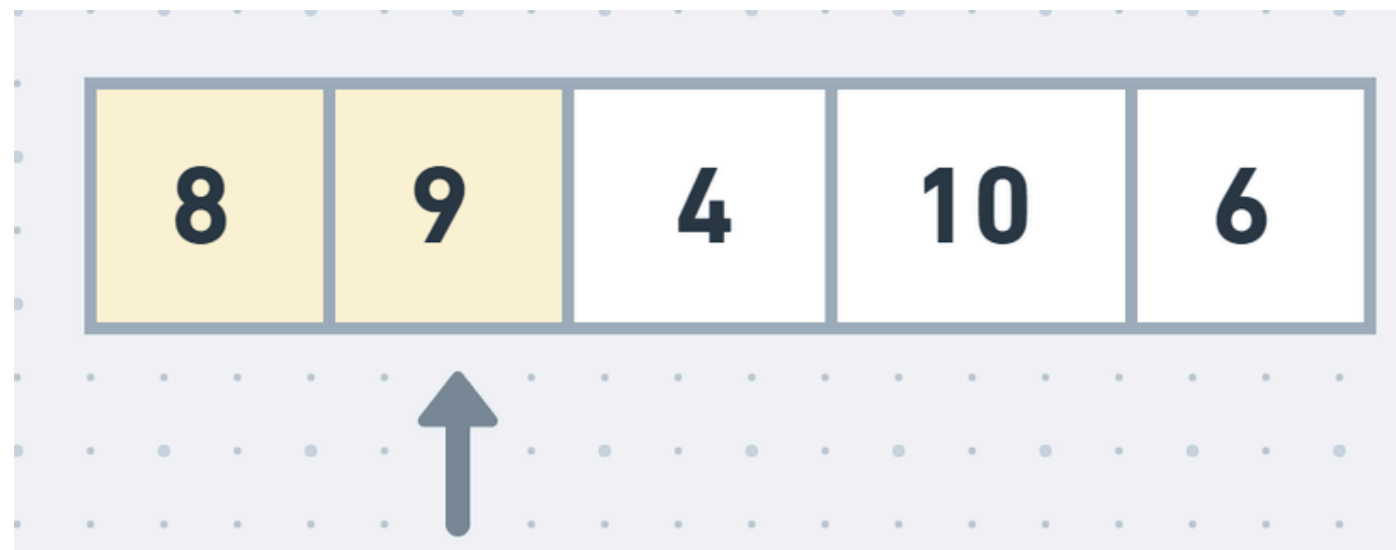
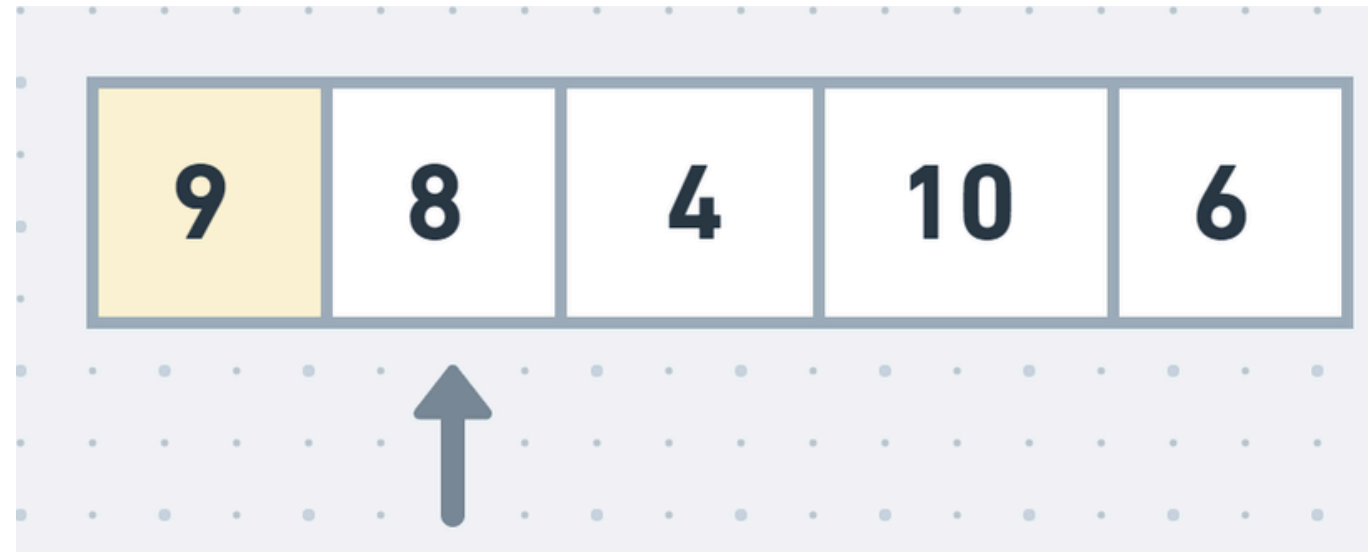
9 8 4 10 6



Insertion Sort

Exemplo: Ordene em ordem crescente

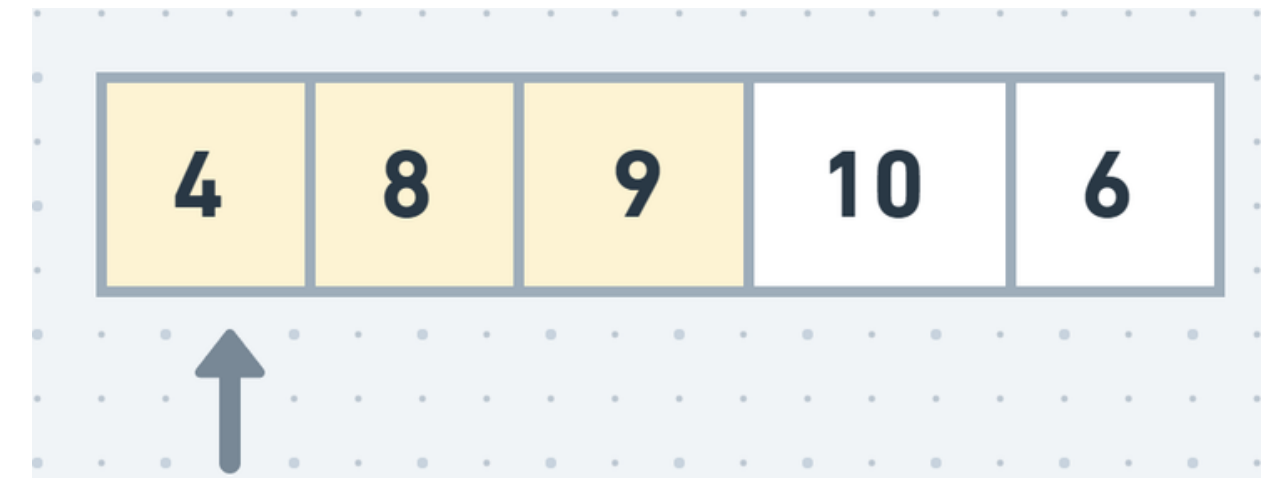
9 8 4 10 6



Insertion Sort

Exemplo: Ordene em ordem crescente

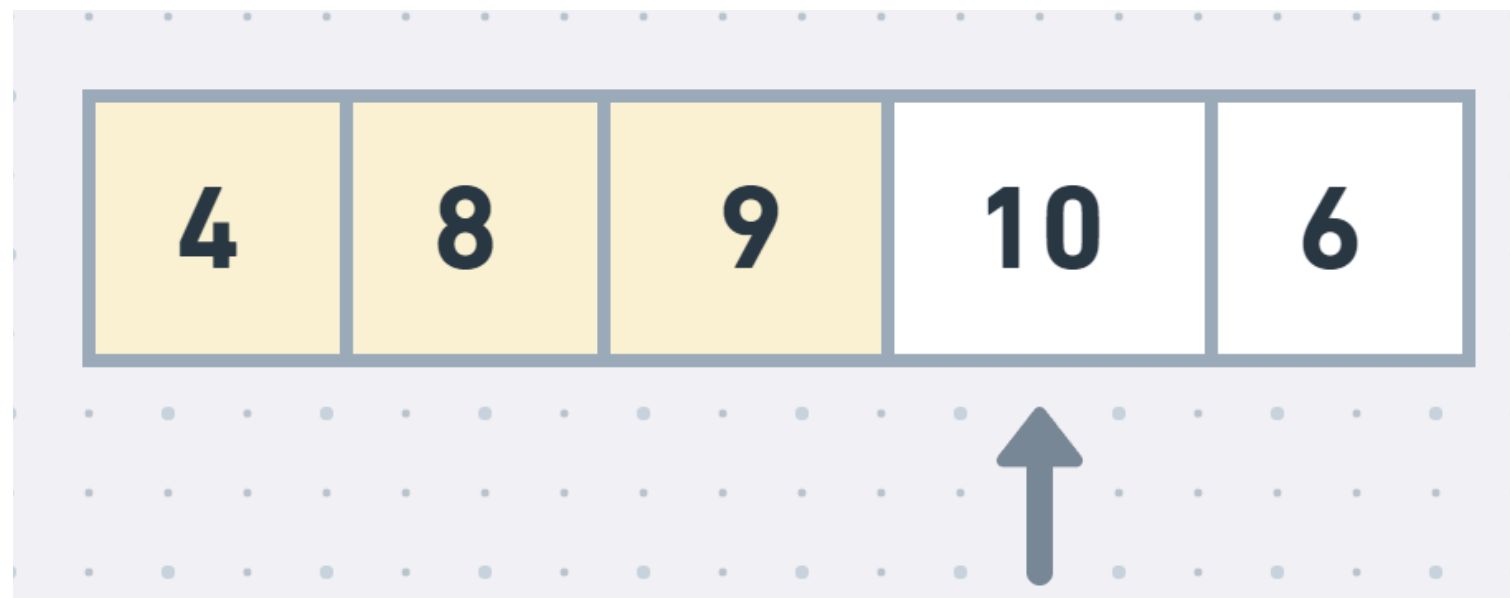
9 8 4 10 6



Insertion Sort

Exemplo: Ordene em ordem crescente

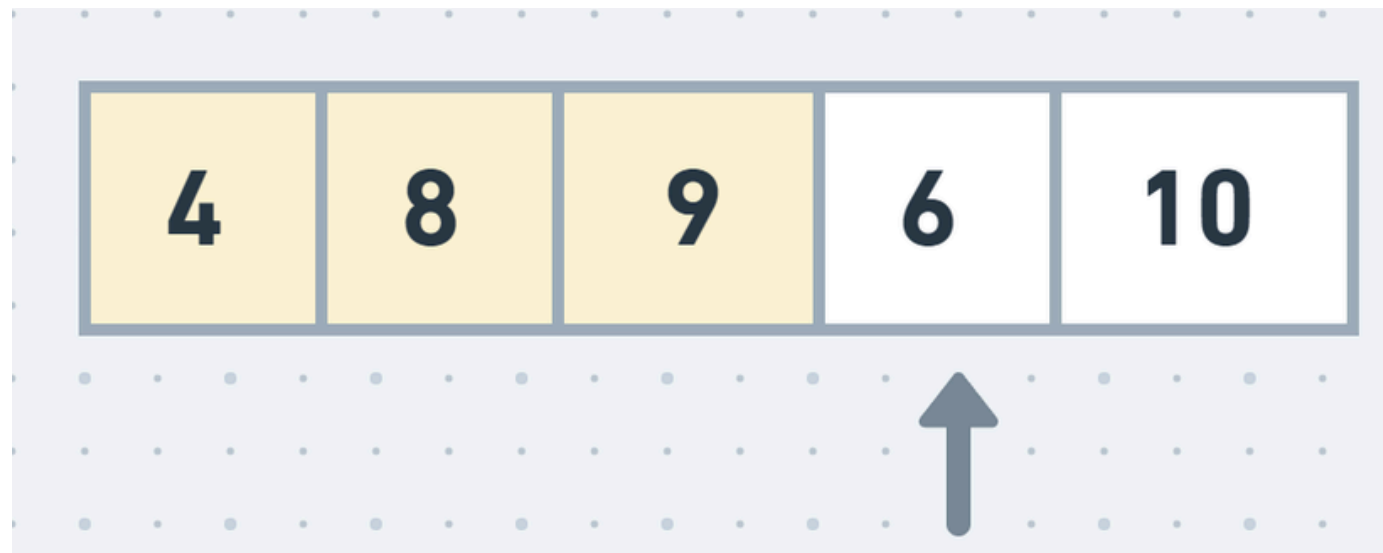
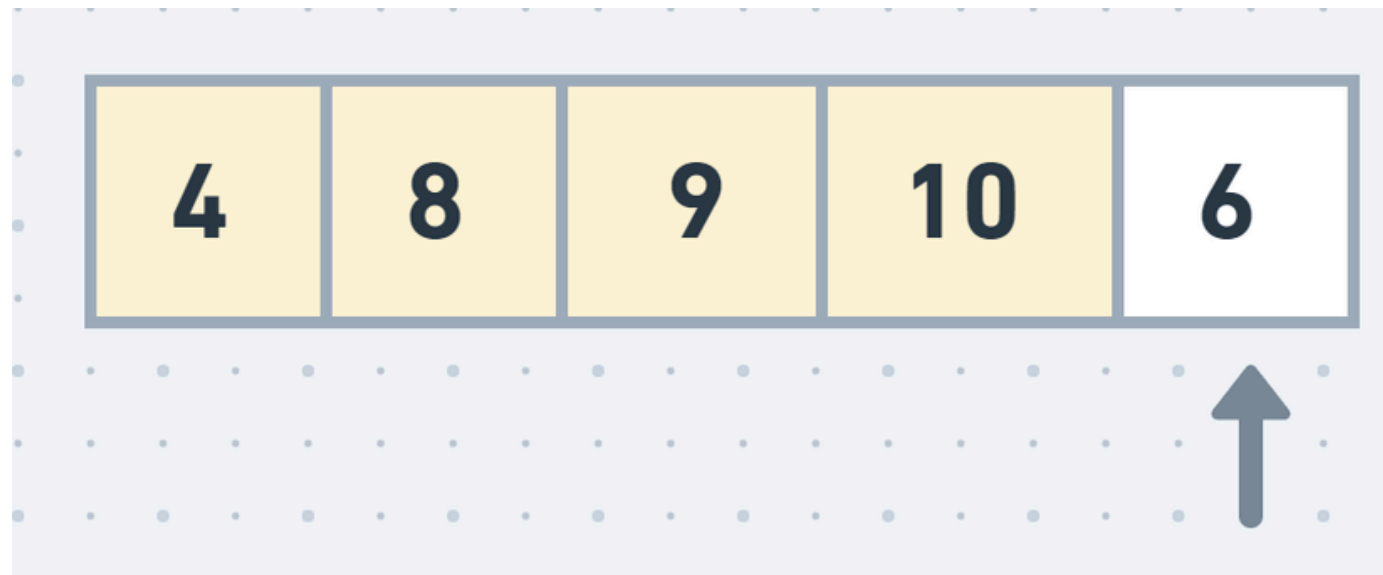
9 8 4 10 6



Insertion Sort

Exemplo: Ordene em ordem crescente

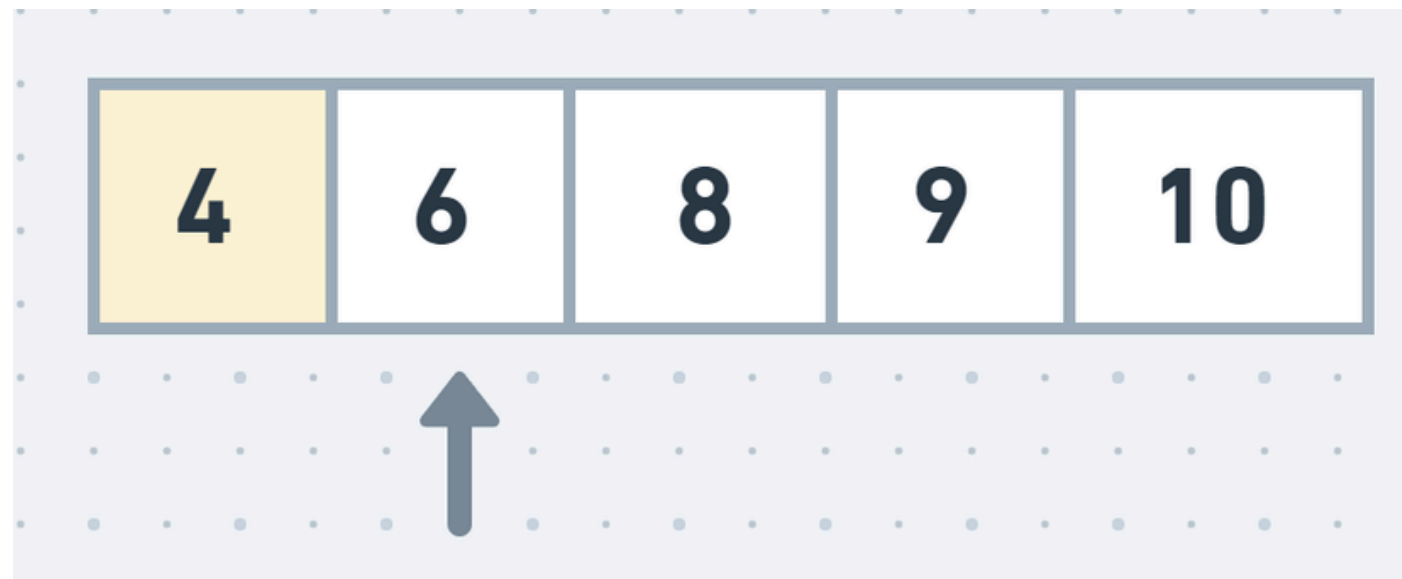
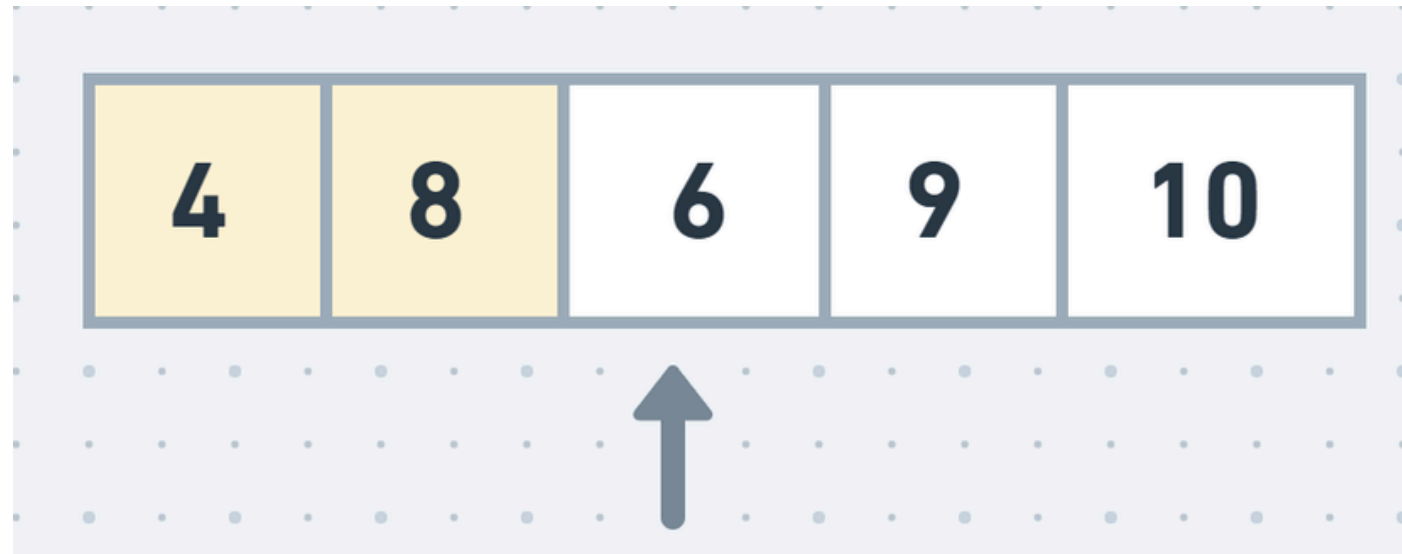
9 8 4 10 6



Insertion Sort

Exemplo: Ordene em ordem crescente

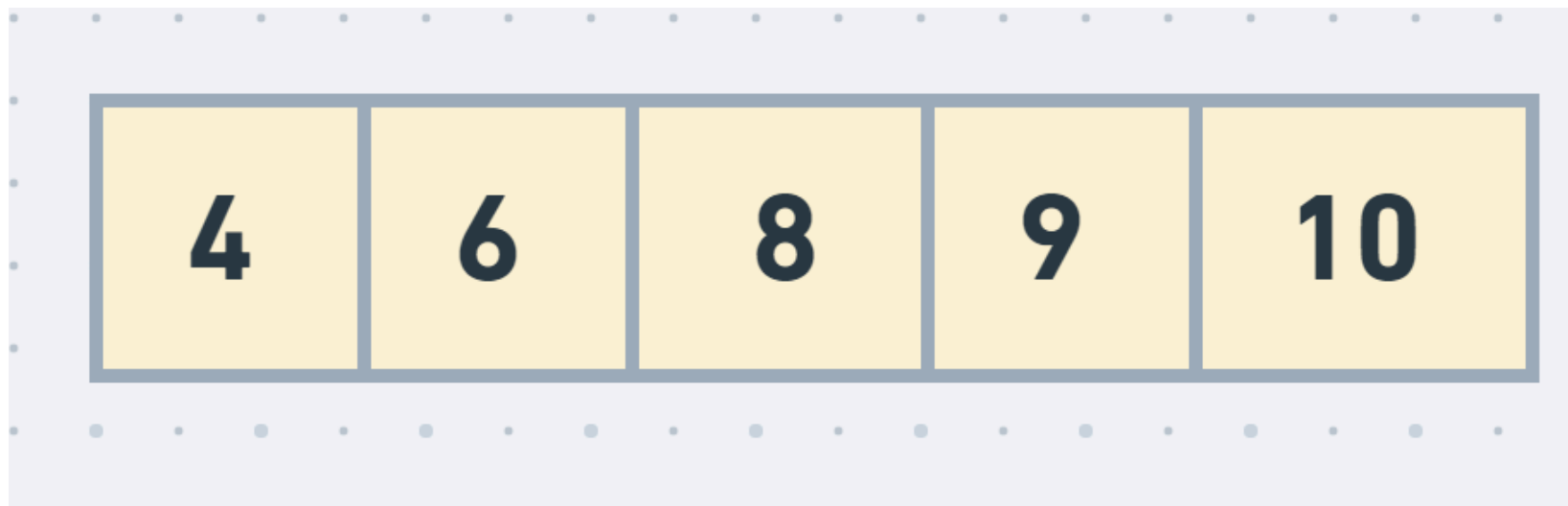
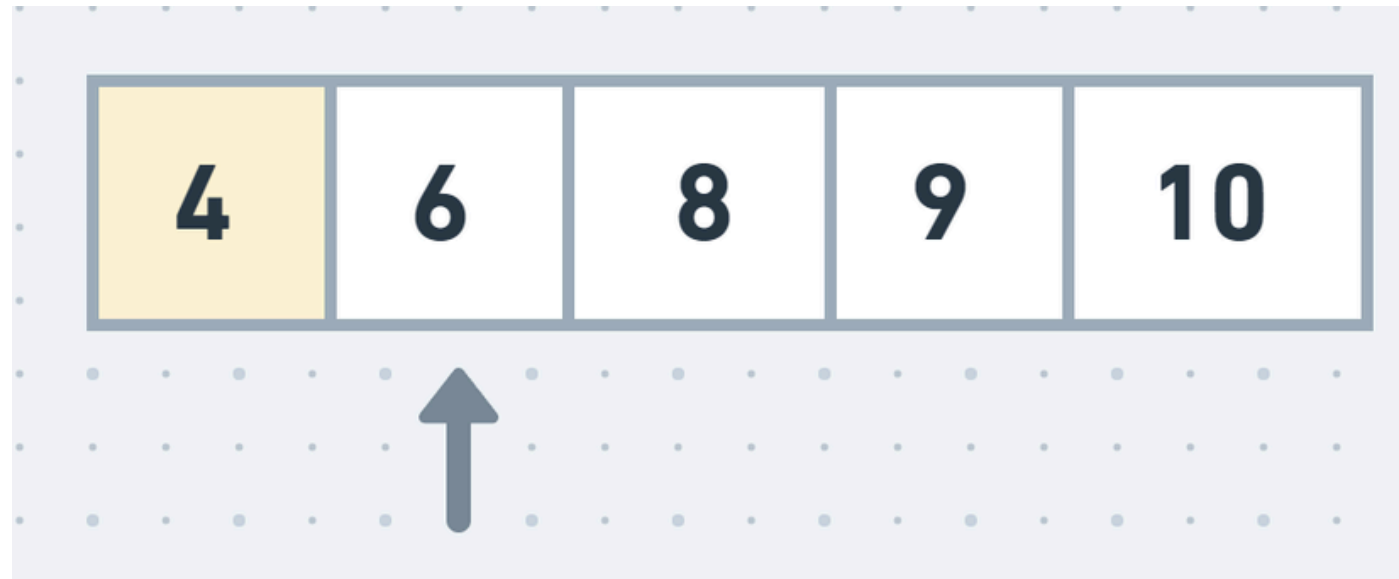
9 8 4 10 6



Insertion Sort

Exemplo: Ordene em ordem crescente

9 8 4 10 6



Insertion Sort



Insertion Sort

Implementem!